

Manual de Instalación Técnico

Sistema AFIP Mobile – Gestión Financiera Personal

Versión: 1.0.0

Fecha: Junio 2026

Tipo de Documento: Manual Técnico

Audiencia: Administradores de sistemas, DevOps, Técnicos de infraestructura

1. Información General

1.1 Propósito

Este manual provee instrucciones técnicas detalladas, scripts de instalación y validaciones para el despliegue completo del sistema **AFIP Mobile** en entorno de producción (Ubuntu Server 22.04 LTS).

1.2 Componentes del Sistema

Componente	Tecnología	Puerto	Directorio
Base de Datos	MySQL 8.0	3306	/var/lib/mysql/
API REST	Node.js 18 + Express 5	3001	/var/www/afip-mobile/api/Node_api/
Frontend Web	React 18 / Nginx	80/443	/var/www/afip-web/
App Mobile	Flutter 3.x	N/A	Distribución via APK/Store
Proxy Reverso	Nginx	80/443	/etc/nginx/
Gestor de Procesos	PM2	–	Global npm

1.3 Dependencias del Sistema

node_project_api (MySQL)

	users	(UUID, user_user UNIQUE)
	accounts	(FK: users)
	categories	(FK: users, ENUM tipo)
	budgets	(FK: users)
	expenses	(FK: users, categories, accounts, budgets)
	incomes	(FK: users, accounts, categories)
	income_assignments	(FK: incomes)
	goals	(FK: users)
	debts	(FK: users)
	debtPayments	(FK: debts)

2. Pre-Instalación

2.1 Verificar Acceso al Servidor

```
# Conectar via SSH
ssh -i ~/.ssh/tu_clave.pem usuario@IP_SERVIDOR

# Verificar sistema operativo
lsb_release -a
# Resultado esperado: Ubuntu 22.04.x LTS

# Verificar espacio en disco disponible
df -h
# Requerido: al menos 20 GB libres en /

# Verificar RAM
free -h
# Requerido: al menos 3.5 GB disponibles
```

2.2 Actualizar el Sistema

```
sudo apt update && sudo apt upgrade -y
sudo apt autoremove -y
```

2.3 Configurar Firewall (UFW)

```
# Habilitar UFW
sudo ufw enable

# Permitir SSH (IMPORTANTE: hacerlo antes de activar UFW)
sudo ufw allow ssh
sudo ufw allow 22/tcp

# Permitir HTTP/HTTPS
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp

# Verificar reglas
sudo ufw status verbose
```

3. Instalación de MySQL 8.0

3.1 Instalar MySQL Server

```
sudo apt install mysql-server -y
sudo systemctl start mysql
sudo systemctl enable mysql

# Verificar estado
sudo systemctl status mysql
# Resultado esperado: ● mysql.service - active (running)
```

3.2 Asegurar la Instalación

```
sudo mysql_secure_installation
```

Responder el asistente:

- `VALIDATE PASSWORD component?` → Y
- `Password validation policy` → 1 (Medium, mínimo recomendado)
- `Change password for root?` → Y → Ingresar contraseña segura

- Remove anonymous users? → Y
- Disallow root login remotely? → Y
- Remove test database? → Y
- Reload privilege tables? → Y

3.3 Crear Usuario y Base de Datos para la Aplicación

```
-- Conectar como root
sudo mysql -u root -p

-- Crear base de datos
CREATE DATABASE IF NOT EXISTS node_project_api
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;

-- Crear usuario de aplicación (NO usar root en producción)
CREATE USER 'afip_user'@'localhost'
  IDENTIFIED WITH mysql_native_password
  BY 'TU_PASSWORD_SEGURA_AQUI';

-- Otorgar permisos
GRANT SELECT, INSERT, UPDATE, DELETE, CREATE, DROP, INDEX, ALTER
  ON node_project_api.*
  TO 'afip_user'@'localhost';

FLUSH PRIVILEGES;

-- Verificar
SHOW GRANTS FOR 'afip_user'@'localhost';
EXIT;
```

3.4 Crear Estructura de Base de Datos

```
# Copiar script SQL al servidor
scp DB/create_database.sql usuario@IP_SERVIDOR:/tmp/

# Ejecutar script
mysql -u afip_user -p node_project_api < /tmp/create_database.sql

# Validar creación de tablas
mysql -u afip_user -p node_project_api -e "SHOW TABLES;"
```

Validación esperada – 10 tablas:

```
+-----+
| Tables_in_node_project_api |
+-----+
| accounts                    |
| budgets                     |
| categories                   |
| debtPayments                 |
| debts                       |
| expenses                     |
| goals                       |
| income_assignments           |
| incomes                     |
| users                       |
+-----+
```

4. Instalación de Node.js y la API

4.1 Instalar Node.js 18 LTS

```
# Repositorio oficial de Nodsource
curl -fsSL https://deb.nodesource.com/setup_18.x | sudo -E bash -
sudo apt install -y nodejs

# Verificar versiones
node --version      # Debe ser v18.x.x
npm --version       # Debe ser 9.x.x o superior
```

4.2 Instalar Git y Clonar el Repositorio

```
sudo apt install git -y
cd /var/www
sudo mkdir -p afip-mobile
sudo chown $USER:$USER afip-mobile
git clone https://github.com/tu-usuario/afip-mobile.git afip-mobile
```

4.3 Instalar Dependencias de la API

```
cd /var/www/afip-mobile/api/Node_api
npm install --omit=dev

# Verificar instalación de módulos clave
ls node_modules | grep -E "express|sequelize|jsonwebtoken|bcryptjs|mysql2"
```

4.4 Configurar Variables de Entorno

```
# Crear archivo .env desde cero
cat > /var/www/afip-mobile/api/Node_api/.env << 'EOF'
SERVER_PORT=3001
DB_NAME=node_project_api
DB_HOST=127.0.0.1
DB_USER=afip_user
DB_PASSWORD=TU_PASSWORD_SEGURA_AQUI
DB_PORT=3306
DB_DIALECT=mysql
JWT_SECRET=REEMPLAZAR_CON_CADENA_ALEATORIA_DE_64_CARACTERES
EOF

# Generar un JWT_SECRET seguro
node -e "console.log(require('crypto').randomBytes(64).toString('hex'))"
# Copiar el resultado y reemplazar JWT_SECRET en .env

# Restringir permisos del archivo .env
chmod 600 /var/www/afip-mobile/api/Node_api/.env
```

4.5 Configurar Sincronización de Modelos

```
# En src/index.js: verificar que modelsApp esté en false para producción
grep "modelsApp" /var/www/afip-mobile/api/Node_api/src/index.js
# Si la DB ya fue creada con el SQL, debe decir: await modelsApp(false);
# Si es la primera vez y no se usó el SQL: usar modelsApp(true) una vez
```

4.6 Instalar PM2 y Configurar el Servicio

```
# Instalar PM2 globalmente
sudo npm install -g pm2

# Iniciar la API con PM2
cd /var/www/afip-mobile/api/Node_api
pm2 start src/index.js --name "afip-api" --interpreter node

# Ver logs en tiempo real
pm2 logs afip-api

# Configurar inicio automático tras reinicio del servidor
pm2 startup systemd
# Ejecutar el comando que PM2 imprime (empieza con "sudo env PATH=...")
pm2 save

# Verificar estado
pm2 status
# Resultado esperado: afip-api | online | 0 restarts
```

4.7 Validar la API

```
# Prueba básica de conectividad
curl -s http://localhost:3001/api/v1/user | python3 -m json.tool
# Esperado: { "message": "No autorizado" } o similar (401)

# Prueba de registro de usuario
curl -s -X POST http://localhost:3001/api/v1/user \
  -H "Content-Type: application/json" \
  -d '{"user_user": "admin@test.com", "user_password": "Admin1234!"}' \
  | python3 -m json.tool
# Esperado: { "ok": true, "status": 200, "token": "..." }

# Prueba de login
curl -s -X POST http://localhost:3001/api/v1/login \
  -H "Content-Type: application/json" \
  -d '{"email": "admin@test.com", "password": "Admin1234!"}' \
  | python3 -m json.tool
# Esperado: { "ok": true, "status": 200, "token": "..." }
```

5. Instalación del Frontend Web

5.1 Instalar Nginx

```
sudo apt install nginx -y
sudo systemctl start nginx
sudo systemctl enable nginx
sudo systemctl status nginx
# Resultado esperado: ● nginx.service - active (running)
```

5.2 Build del Frontend Web

```
# En la máquina de desarrollo:
cd front-end-web/finance-frontend-react

# Crear .env de producción
echo "VITE_API_URL=https://tu-dominio.com/api/v1" > .env.production

# Instalar y compilar
npm install
npm run build

# Transferir archivos compilados
scp -r dist/ usuario@IP_SERVIDOR:/var/www/afip-web/
```

5.3 Configurar Nginx

```
# Crear configuración del sitio
sudo nano /etc/nginx/sites-available/afip-mobile
```

Contenido completo:

```
server {
    listen 80;
    server_name tu-dominio.com www.tu-dominio.com;

    # Frontend Web
    root /var/www/afip-web;
    index index.html;

    # SPA routing
    location / {
        try_files $uri $uri/ /index.html;
    }
}
```



```

        add_header Cache-Control "no-cache, no-store, must-revalidate";
    }

    # Archivos estáticos con caché
    location ~* \.(js|css|png|jpg|jpeg|gif|ico|svg)$ {
        expires 1y;
        add_header Cache-Control "public, immutable";
    }

    # Proxy a la API Node.js
    location /api/ {
        proxy_pass http://localhost:3001;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_cache_bypass $http_upgrade;
    }
}

```

```

# Activar el sitio
sudo ln -s /etc/nginx/sites-available/afip-mobile /etc/nginx/sites-enabled/

# Deshabilitar el sitio por defecto
sudo rm /etc/nginx/sites-enabled/default

# Verificar sintaxis
sudo nginx -t
# Resultado: nginx: the configuration file /etc/nginx/nginx.conf syntax is ok

# Recargar Nginx
sudo systemctl reload nginx

```

5.4 Configurar HTTPS con Let's Encrypt

```

sudo apt install certbot python3-certbot-nginx -y
sudo certbot --nginx -d tu-dominio.com -d www.tu-dominio.com

# Renovación automática (ya configurada por certbot)
sudo certbot renew --dry-run # Prueba de renovación

```

6. Instalación de la Aplicación Mobile

6.1 Instalar Flutter SDK (en máquina de desarrollo)

```
# Descargar Flutter
wget https://storage.googleapis.com/flutter_infra_release/releases/stable/linux/flutter_linux_3.x.x-stable.tar.xz

# Descomprimir
tar xf flutter_linux_3.x.x-stable.tar.xz -C ~/development/

# Agregar al PATH
echo 'export PATH="$PATH:$HOME/development/flutter/bin"' >> ~/.bashrc
source ~/.bashrc

# Verificar
flutter --version
flutter doctor
```

6.2 Compilar la Aplicación Android

```
cd front-end-mobile/application_login

# Obtener dependencias
flutter pub get

# Compilar APK de release
flutter build apk --release

# O compilar App Bundle para Play Store
flutter build appbundle --release

# Ubicación del APK generado:
# build/app/outputs/flutter-apk/app-release.apk
```

7. Comandos de Gestión Post-Instalación

Gestión de la API (PM2)

```
pm2 status          # Ver estado de todos los procesos
pm2 restart afip-api # Reiniciar la API
pm2 stop afip-api    # Detener la API
pm2 logs afip-api    # Ver logs en tiempo real
pm2 logs afip-api --lines 100 # Ver últimas 100 líneas de log
pm2 monit            # Monitor en tiempo real
```

Gestión de MySQL

```
sudo systemctl status mysql # Estado del servicio
sudo systemctl restart mysql # Reiniciar MySQL
mysql -u afip_user -p node_project_api # Conectar a la BD
```

Gestión de Nginx

```
sudo nginx -t          # Verificar sintaxis de config
sudo systemctl reload nginx # Recargar sin interrumpir
sudo systemctl restart nginx # Reiniciar completamente
sudo journalctl -u nginx -f # Ver logs de Nginx
```

8. Validación Final de Instalación

Ejecutar el siguiente script de validación:

```
#!/bin/bash
echo "=====
echo " AFIP Mobile - Validación de Instalación "
echo "=====

# 1. MySQL
echo -n "[1] MySQL activo: "
systemctl is-active mysql && echo "✅ OK" || echo "❌ FALLO"

# 2. Tablas en BD
echo -n "[2] Tablas creadas (10): "
```

```

COUNT=$(mysql -u afip_user -pTU_PASSWORD node_project_api -e "SELECT COUNT(*) FROM informat
[ "$COUNT" = "10" ] && echo "✅ OK ($COUNT tablas)" || echo "❌ FALLO ($COUNT tablas)"

# 3. API Node.js
echo -n "[3] API corriendo (PM2): "
pm2 list | grep afip-api | grep -q online && echo "✅ OK" || echo "❌ FALLO"

# 4. API responde HTTP
echo -n "[4] API responde en :3001: "
STATUS=$(curl -s -o /dev/null -w "%{http_code}" http://localhost:3001/api/v1/user)
[ "$STATUS" = "401" ] && echo "✅ OK (HTTP $STATUS - protegido)" || echo "❌ FALLO (HTTP $S

# 5. Nginx activo
echo -n "[5] Nginx activo: "
systemctl is-active nginx && echo "✅ OK" || echo "❌ FALLO"

# 6. Puerto 80 accesible
echo -n "[6] Puerto 80 accesible: "
STATUS=$(curl -s -o /dev/null -w "%{http_code}" http://localhost)
[ "$STATUS" = "200" ] && echo "✅ OK" || echo "⚠️ HTTP $STATUS"

echo "=====
echo " Validación completada"
echo "=====

```

9. Solución de Problemas Técnicos

Error	Diagnóstico	Solución
Error: Cannot find module 'sequelize'	npm install no ejecutado	cd api/Node_api && npm install
SequelizeConnectionError: ECONNREFUSED	MySQL detenido o credenciales incorrectas	Verificar MySQL activo y variables .env
JsonWebTokenError: invalid signature	JWT_SECRET diferente entre registros	Asegurar JWT_SECRET consistente entre deploys
SyntaxError: Cannot use import...	Node.js < 18 o "type": "module" faltante	Verificar Node 18+ y package.json
nginx: [emerg] bind() to 0.0.0.0:80 failed	Puerto 80 ocupado por otro proceso	sudo lsof -i :80 y terminar el proceso
Error: EACCES: permission denied	Permisos insuficientes en directorio	sudo chown -R \$USER:\$USER /var/www/afip-mobile

<code>flutter pub get</code> falla	SDK desactualizado o sin internet	<code>flutter upgrade && flutter pub cache repair</code>
------------------------------------	-----------------------------------	--